

```
In [1]: ##### Import python libraries
import psycopg2
```

```
In [2]: import pandas as pd
```

```
In [3]: df = pd.read_csv(r"C:\Users\hp\Desktop\Datasets\sales.csv")
```

Extract

```
In [4]: def Extract():
        df = pd.read_csv(r"C:\Users\hp\Desktop\Datasets\sales.csv")
        return df
```

Transform

```
In [5]: df.duplicated().sum()
```

```
Out[5]: np.int64(0)
```

```
In [6]: def Transform():
        df.drop('ADDRESSLINE2',axis = 1, inplace = True)
        return df
```

```
In [7]: df[df['STATE'].isnull()].head()
```

Out[7]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE
1	10121	34	81.35	5	2765.90	5/7/11
2	10134	41	94.74	2	3884.34	7/1/11
6	10180	29	86.13	9	2497.77	11/11/11
7	10188	48	100.00	1	5512.32	11/18/11
9	10211	41	100.00	14	4708.44	1/15/12

5 rows × 7 columns



```
In [8]: df[df['STATE'].notnull()].head()
```

Out[8]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE
0	10107	30	95.70	2	2871.00	2/24/2005
3	10145	45	83.26	6	3746.70	8/25/2005
4	10159	49	100.00	14	5205.27	10/10/2005
5	10168	36	96.66	1	3479.76	10/28/2005
8	10201	22	98.57	2	2168.54	12/14/2005

5 rows × 25 columns

In [9]:

```
df.groupby('COUNTRY')['STATE'].apply(lambda x: x.isnull().mean())
```

Out[9]:

COUNTRY	
Australia	0.000000
Austria	1.000000
Belgium	1.000000
Canada	0.000000
Denmark	1.000000
Finland	1.000000
France	1.000000
Germany	1.000000
Ireland	1.000000
Italy	1.000000
Japan	0.000000
Norway	1.000000
Philippines	1.000000
Singapore	1.000000
Spain	1.000000
Sweden	1.000000
Switzerland	1.000000
UK	0.819444
USA	0.000000

Name: STATE, dtype: float64

In [10]:

```
df['STATE'] = df['STATE'].fillna('Unknown')
```

In [11]:

```
print(df.isnull().sum())
```

ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	2521
CITY	0
STATE	0
POSTALCODE	76
COUNTRY	0
TERRITORY	1074
CONTACTLASTNAME	0
CONTACTFIRSTNAME	0
DEALSIZE	0

dtype: int64

```
In [12]: pd.crosstab(df['COUNTRY'], df['TERRITORY'])
```

Out[12]:

TERRITORY	APAC	EMEA	Japan
COUNTRY			
Australia	185	0	0
Austria	0	55	0
Belgium	0	33	0
Denmark	0	63	0
Finland	0	92	0
France	0	314	0
Germany	0	62	0
Ireland	0	16	0
Italy	0	113	0
Japan	0	0	52
Norway	0	85	0
Philippines	0	0	26
Singapore	36	0	43
Spain	0	342	0
Sweden	0	57	0
Switzerland	0	31	0
UK	0	144	0

In [13]:

df[df['COUNTRY'] == 'Singapore']

Out[13]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	O
30	10150	45	100.00	8	10993.50	
137	10150	20	100.00	1	3191.20	
143	10217	48	100.00	4	7020.48	
146	10259	26	100.00	12	4033.38	
187	10117	33	100.00	9	6034.38	
...	...	...	...	...	...	...
2510	10117	38	79.68	6	3027.84	
2618	10150	20	100.00	3	2104.00	
2624	10217	31	88.00	6	2728.00	
2718	10117	45	83.42	1	3753.90	
2798	10117	50	43.68	2	2184.00	

79 rows × 25 columns

In [14]:

```
df[df['TERRITORY'].isnull()] ['COUNTRY'].value_counts()
```

Out[14]: COUNTRY
USA 1004
Canada 70
Name: count, dtype: int64

In [15]:

```
pd.crosstab(df['COUNTRY'], df['TERRITORY'], dropna=False)
```

Out[15]:

TERRITORY	APAC	EMEA	Japan	NaN
COUNTRY				
Australia	185	0	0	0
Austria	0	55	0	0
Belgium	0	33	0	0
Canada	0	0	0	70
Denmark	0	63	0	0
Finland	0	92	0	0
France	0	314	0	0
Germany	0	62	0	0
Ireland	0	16	0	0
Italy	0	113	0	0
Japan	0	0	52	0
Norway	0	85	0	0
Philippines	0	0	26	0
Singapore	36	0	43	0
Spain	0	342	0	0
Sweden	0	57	0	0
Switzerland	0	31	0	0
UK	0	144	0	0
USA	0	0	0	1004

```
In [16]: df.loc[
    (df['COUNTRY'].isin(['USA', 'Canada'])) &
    (df['TERRITORY'].isna()), 'TERRITORY'] = 'North America'
```

```
In [17]: df[df['COUNTRY'].isin(['USA', 'Canada'])][['COUNTRY', 'TERRITORY']].head(20)
```

Out[17]:

	COUNTRY	TERRITORY
0	USA	North America
3	USA	North America
4	USA	North America
5	USA	North America
8	USA	North America
11	USA	North America
12	USA	North America
13	USA	North America
15	USA	North America
18	USA	North America
19	USA	North America
23	USA	North America
29	USA	North America
31	USA	North America
33	USA	North America
35	Canada	North America
36	USA	North America
37	USA	North America
38	USA	North America
44	USA	North America

```
In [18]: df.loc[
          df['COUNTRY'].isin(['USA', 'Canada']), 'TERRITORY'].isna().sum()
```

Out[18]: np.int64(0)

```
In [19]: df[df['TERRITORY'].isna()] ['COUNTRY'].value_counts()
```

Out[19]: Series([], Name: count, dtype: int64)

```
In [20]: df['TERRITORY'].isna().sum()
```

Out[20]: np.int64(0)

```
In [21]: df[df['COUNTRY'].isin(['USA', 'Canada'])] ['TERRITORY'].value_counts(dropna=False)
```

```
Out[21]: TERRITORY
North America    1074
Name: count, dtype: int64
```

```
In [22]: df[df['TERRITORY'].isna()] ['COUNTRY'].value_counts()
```

```
Out[22]: Series([], Name: count, dtype: int64)
```

```
In [23]: df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'])
```

```
In [24]: df['Total_Revenue'] = df['QUANTITYORDERED'] * df['PRICEEACH']
```

```
In [25]: df = df[df['YEAR_ID'] == 2004]
```

```
In [26]: df.rename(columns={'Total_Revenue': 'TOTAL_REVENUE'}, inplace=True)
```

```
In [27]: df.columns
```

```
Out[27]: Index(['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER',
               'SALES', 'ORDERDATE', 'STATUS', 'QTR_ID', 'MONTH_ID', 'YEAR_ID',
               'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE',
               'ADDRESSLINE1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE',
               'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRSTNAME',
               'DEALSIZE', 'TOTAL_REVENUE'],
              dtype='object')
```

```
In [28]: pd.set_option('display.max_columns', None)
df.head()
```

```
Out[28]:
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE
9	10211	41	100.0	14	4708.44	2004-01-01
10	10223	37	100.0	1	3965.66	2004-01-01
11	10237	23	100.0	7	2333.12	2004-01-01
12	10251	28	100.0	2	3188.64	2004-01-01
13	10263	34	100.0	2	3676.76	2004-01-01

9	10211	41	100.0	14	4708.44	2004-01-01
10	10223	37	100.0	1	3965.66	2004-01-01
11	10237	23	100.0	7	2333.12	2004-01-01
12	10251	28	100.0	2	3188.64	2004-01-01
13	10263	34	100.0	2	3676.76	2004-01-01



LOAD

```
In [34]: fil_path = "sales.csv"

def Extract():
    df = pd.read_csv(r"C:\Users\hp\Desktop\Datasets\sales.csv")
```



```

    return df

def Transform(df):
    df.drop('ADDRESSLINE2',axis = 1, inplace = True)
    df['STATE'] = df['STATE'].fillna('Unknown')
    df.loc[
        (df['COUNTRY'].isin(['USA', 'Canada'])) &
        (df['TERRITORY'].isna()), 'TERRITORY'] = 'North America'
    df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'])
    df['Total_Revenue'] = df['QUANTITYORDERED'] * df['PRICEEACH']
    df = df[df['YEAR_ID'] == 2004]
    df.rename(columns={'Total_Revenue': 'TOTAL_REVENUE'}, inplace=True)
    return df

def load(df):
    connection = psycopg2.connect(
        host = "localhost",
        dbname = "Sales ",
        user = "postgres",
        password = "1234"

    )

    con = connection.cursor()

    for _, row in df.iterrows():
        con.execute("""
            INSERT INTO Sales_Record(
                ORDERNUMBER, QUANTITYORDERED, PRICEEACH, ORDERLINENUMBER,
                SALES, ORDERDATE, STATUS, QTR_ID, MONTH_ID, YEAR_ID,
                PRODUCTLINE, MSRP, PRODUCTCODE, CUSTOMERNAME, PHONE,
                ADDRESSLINE1, CITY, STATE, POSTALCODE, COUNTRY, TERRITORY,
                CONTACTLASTNAME, CONTACTFIRSTNAME, DEALSIZE, TOTAL_REVENUE)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
            ON CONFLICT (ORDERNUMBER) DO NOTHING """, tuple (row)
        )

    connection.commit()
    con.close()
    connection.close()

if __name__ == "__main__":
    try:
        raw_data = Extract()
        clean_data = Transform(raw_data)
        load(clean_data)
    except Exception as e:
        print(f"Error: {e}")

```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_14628\1651861553.py:16: SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
df.rename(columns={'Total_Revenue': 'TOTAL_REVENUE'}, inplace=True)
```

```
In [30]: print(df.columns.tolist())
```

```
['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER', 'SALES', 'ORDERDATE', 'STATUS', 'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE', 'ADDRESSLINE1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE', 'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRSTNAME', 'DEALSIZE', 'TOTAL_REVENUE']
```

```
In [ ]:
```